

★ Semantic Analysis

* Syntax Directed Translation

↳ SDT has augmented rules to the grammar that facilitate semantic analysis.

↳ CFG + Semantic rules/actions

Ex:

$S \rightarrow A B \{ \text{print}(\ast) \}^{(1)}$

$A \rightarrow a \{ \text{print}(a) \}^{(2)}$

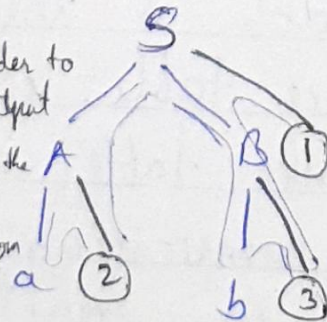
$B \rightarrow b \{ \text{print}(b) \}^{(3)}$

actions/
semantic rules

Input $w = ab$

Output:
 $ab \ast$

Now, in order to generate output we'll parse the parse tree top to bottom & left to right



→ In addition to parse tree through TOP $\longleftrightarrow$ we'll add one extra node according to semantic action



⇒ What can be done using SOT?

- Tune - checking

- Type checking
- Algebraic expression evaluation
- Syntax tree
- Verification of variable declaration

In short, SOT helps for semantic analysis tasks.

$\Rightarrow$ Annotated Parse tree and attributes:

↳ The parse tree containing the values of attributes at each node for given input string is called as annotated or decorated parse tree.

↳ Attributes are related to the grammar symbol that are the labels of the parse tree node.

Ex: Gr: $E \rightarrow E + E \quad \{ E.val = E.val + E.val \}^{(1)}$ val is
 $E \rightarrow E * E \quad \{ E.val = E.val * E.val \}^{(2)}$ attribute here
 $E \rightarrow id \quad \{ E.val = id.val \}^{(3)}$
 CFCG action

Input: $w = 2 + 3 * 5$

Value of each id
get stored during lex.
analysis in symbol
table

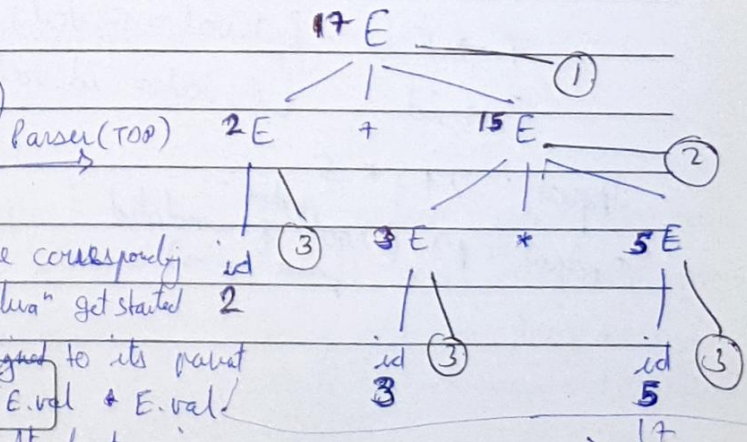
lex. analysis (symbol table)

$$w = id + id * id$$

- All id in parse table will get the corresponding value. - Through each action the evaluation get started

as ③ E.val = id.val 2 will get assigned to its parent
similarly 3 & 5, then using ② E.val = E.val + E.val

15 will get assigned to its parent E. At last using
 ① $E.val = E.val + E.val = 2 + 15 = 17$ will get assigned to parent/root $\Rightarrow$ 17 as output



- Type checking

- Algebraic expression evaluation

- Syntax tree

- Verification of Variable declaration

In short, SOT helps for semantic analysis tasks.

↳ The parse tree containing the values of attributes at each node for given input string is called as annotated or decorated parse tree.

↳ Attributes are related to the grammar symbol that are the labels of the parse tree node.

Ex: Gr: $E \rightarrow E + E$ { $E.val = E.val + E.val$ }⁽¹⁾ val is
 $E \rightarrow E * E$ { $E.val = E.val * E.val$ }⁽²⁾ attribute here
 $E \rightarrow id$ { $E.val = id.val$ }⁽³⁾
 CFCG action

Input: $w = 2 + 3 * 5$

Value of each id
get stored during lex.
analysis in symbol
table

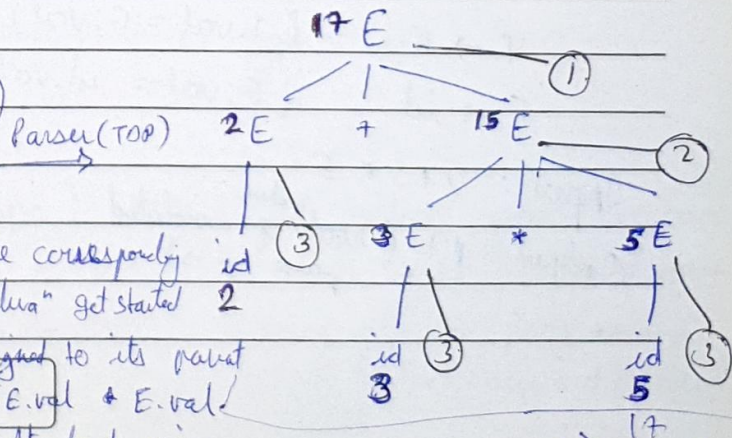
lex. analysis (Symbol
table)

$w = id + id * id$

- All id in parse table will get the corresponding value. - Through each action the evaluation get started 2

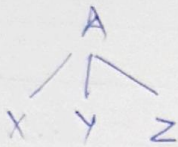
as ③ E.val = id.val 2 will get assigned to its parent
similarly 3 & 5, then using ② E.val = E.val + E.val

13 will get assigned to its parent E. At last using
 ① $E.val = E.val + E.val = 2 + 15 = 17$ will get assigned to parent/root $\Rightarrow$ as output



Types of attributes : Synthesized & Inherited

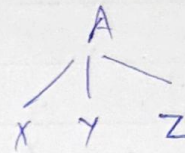
→ $A \rightarrow XYZ$



$A.S = f(X.S, Y.S, Z.S)$ ^{i.e. children only}
attribute S is synthesized

→ These are attributes which derive their values from their children nodes i.e. value of synthesized attribute at node is computed from values of attributes from children nodes in parse tree

$A \rightarrow XYZ$



$X.S = f(Y.S, Z.S, A.S)$

i.e. parent or/and sibling attribute S is inherited

→ These are the attributes which derive their values from their parent or sibling nodes i.e. value of inherited attributes are computed by value of parent or sibling nodes.

Examples:

1) Construct SDT for expression evaluation

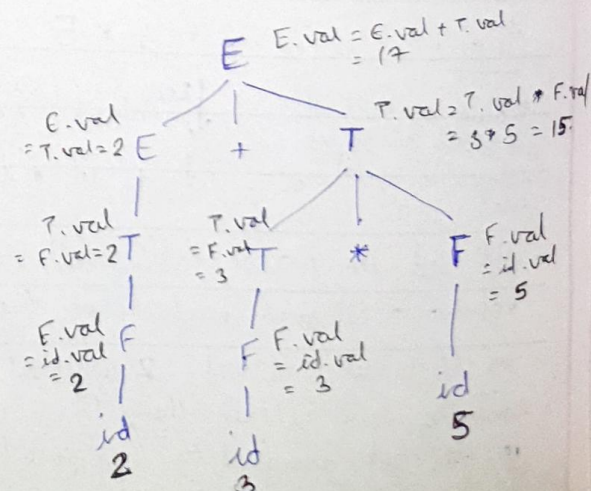
Gr:

- $E \rightarrow E + T \quad \{ E.val = E.val + T.val \}$
- $E \rightarrow T \quad \{ E.val = T.val \}$
- $T \rightarrow T * F \quad \{ T.val = T.val * F.val \}$
- $T \rightarrow F \quad \{ T.val = F.val \}$
- $F \rightarrow id \quad \{ F.val = id.val \}$

$W = 2 + 3 * 5$

↓ LA (+symbol Table)

$id + id * id$



Input : $2 + 3 * 5$

Output : 17 (root value of annotated parse tree)





2) SDT for infix to postfix conversion

$E \rightarrow E + T \quad \{ E.val = E.val + T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow T * F \quad \{ T.val = T.val * F.val \}$

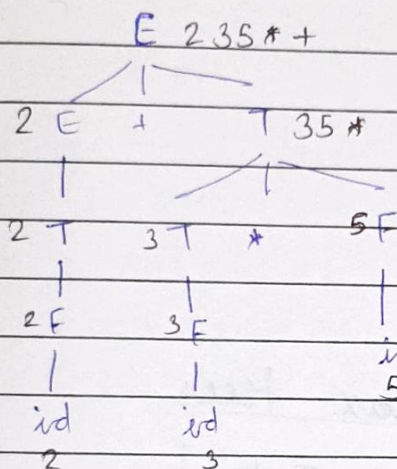
$T \rightarrow F \quad \{ T.val = F.val \}$

$F \rightarrow id \quad \{ F.val = id.val \}$

Input: 2 + 3 * 5

EA + Symbol
table
↓

id + id * id



⇒ Output: 2 3 5 * +

3) SDT for type-checking

$E \rightarrow E + T \quad \{ \text{if } (E.type == T.type) \wedge (T.type == int) \text{ then } E.type = int \text{ else ERROR} \}$

$E \rightarrow T \quad \{ E.type = T.type \}$

$T \rightarrow T * F \quad \{ \text{if } (T.type == F.type) \wedge (F.type == int) \text{ then } T.type = int \text{ else ERROR} \}$

$T \rightarrow F \quad \{ T.type = F.type \}$

$F \rightarrow id \quad \{ F.type = id.type \}$

For verification:

as an
int variable
in w_1, w_2
 w_3

$w_1 = 2 + 3 + 5$ ✓

$w_2 = 2 + 3 * 5$ ✓

$w_3 = 2 + 3 * 2$ ✗ Error

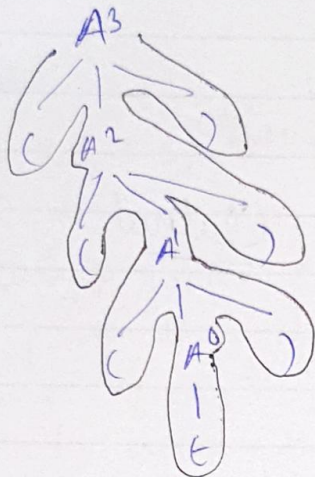
In action part we can write our desired programming or non-programming stuff which carry out desired result.

4) SDT to count parenthesis

Cr: $A \rightarrow (A) \{ A.count = A.count + 1 \}$
 $A \rightarrow \epsilon \{ A.count = 0 \}$

Input : $((()))$

Output : 3



5) SDT to construct syntax trees

$E \rightarrow E + T \{ E.ptr = \text{mknode}(E.ptr, +, T.ptr) \}$

$E \rightarrow T \{ E.ptr = T.ptr \}$

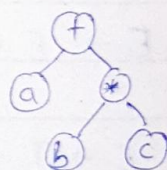
$T \rightarrow T * F \{ T.ptr = \text{mknode}(T.ptr, *, F.ptr) \}$

$T \rightarrow F \{ T.ptr = F.ptr \}$

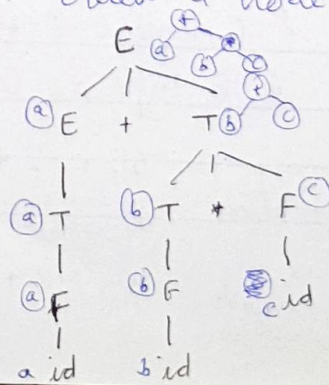
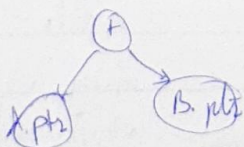
$F \rightarrow \text{id} \{ F.ptr = \text{mknode}(\text{NULL}, \text{id}, \text{NULL}) \}$

Input:
 $a + b * c$

Output:



Here $\text{mknode}()$ is a function which'll create a node as $\text{mknode}(A.ptr, +, B.ptr)$



$\Rightarrow$ Types of SDT : S-attributed & L-attributed

S-attributed SDT

① Only uses synthesized attributes

2) Actions are placed at the end of RHS.

Ex: $S \rightarrow AB \{ \text{print } * \}$
 $A \rightarrow a \{ \text{print } a \}$
 $B \rightarrow b \{ \text{print } b \}$

3) Attributes can be evaluate using top-down or bottom-up methods

L-attributed SDT

① It uses both synthesized & inherited attributes
 (But inherited attr. are restricted to using attr. from parent & left siblings only).

2) Actions can be placed anywhere on RHS.

Ex: $S \rightarrow A \{ \text{print } * \} B$
 $A \rightarrow \{ \text{print } a \} a$
 $B \rightarrow b \{ \text{print } b \}$

3) Attributes can be evaluate using depth-first (top-down) L to R.

Note: Every S-attributed SDT is an L-attributed SDT

- An L-attributed SDT can sometime convert into S-attributed SDT as,

$S \rightarrow A \textcircled{1} B$
 $A \rightarrow \textcircled{2} a$
 $B \rightarrow b \textcircled{3}$

$\Rightarrow$

$S \rightarrow ARB \{ \}$
 $R \rightarrow \epsilon \textcircled{1}$
 $A \rightarrow Xa \{ \}$
 $X \rightarrow \epsilon \textcircled{2}$
 $B \rightarrow b \textcircled{3}$

We can have an empty actions

